

Git

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Brief overview / introduction	2
2	Repository: Directory content tracking	23

Brief overview / introduction

- **Local vs remote repos**
 - A **local repository** is a repository that is stored on a computer.
 - A **remote repository** is a repository that is hosted on a hosting service.
- **Local repositories:** A local repository is represented by a hidden directory called `.git` that exists within a project directory. It contains all the data on the changes that have been made to the files in a project.

You should never touch any files or directories inside your `.git` directory. Doing this could have undesirable consequences for your repository. You should never delete this directory unless you want to delete your repository.

- **git init:** `git init` creates a new empty Git repository, or reinitializes an existing one. In the normal case, it creates a `.git/` directory containing Git's object database, refs, tags, config, hooks/templates, and other repository metadata. Running `git init` again inside an existing repository is safe; Git will not overwrite existing repository data.

```
1  git init [-q | --quiet]
2          [--bare]
3          [--template=<template-directory>]
4          [--separate-git-dir <git-dir>]
5          [--object-format=<format>]
6          [--ref-format=<format>]
7          [-b <branch-name> | --initial-branch=<branch-name>]
8          [--shared[=<permissions>]]
9          [<directory>]
```

- `-q` Suppress normal output. Git will only print errors and warnings. This is useful in scripts where you do not want the usual “Initialized empty Git repository...” message.
- `--bare` Creates a bare repository. A bare repository has no working tree; it is essentially just the repository database itself. Instead of storing Git metadata inside `.git/`, the repository directory itself contains things like `objects/`, `refs/`, `HEAD`, and `config`. Git's repository-layout documentation describes a bare repository as one “without its own working tree,” typically used for exchanging histories by pushing and fetching.
- `--template` Tells Git where to copy template files from when initializing the repository. Templates can include default hook samples, info files, exclude patterns, and initial repository structure. Git chooses the template directory in this order: the `--template` argument, then `GIT_TEMPLATE_DIR`, then `init.templateDir`, then the default template directory.

This is useful if you want every new repo to start with the same hooks or config-related files.

- `--separate-git-dir` Stores the actual Git repository data somewhere other than `.git/` in the working directory. Instead of creating a real `.git/` directory, Git creates a `.git` text file that points to the actual repository directory. If used while reinitializing, Git moves the repository to the specified path

Stores the actual Git repository data somewhere other than `.git/` in the working directory. Instead of creating a real `.git/` directory, Git creates a `.git` text file that points to the actual repository directory. If used while reinitializing, Git moves the repository to the specified path

- `-object-format` Chooses the hash algorithm used for Git objects. Valid values are currently `sha1` and, if supported by your Git build, `sha256`. `sha1` is the default. Git’s documentation also notes that SHA-256 repositories and SHA-1 repositories currently do not interoperate directly.
- `-ref-format` Chooses how Git stores refs, such as branches and tags. Valid values are `files` and `reftable`. The default is `files`, meaning loose ref files plus packed-refs. `reftable` uses the newer reftable storage format.
- `-b` Sets the name of the initial branch in the new repository. These two forms are equivalent. For example, this creates the repo with `main` as the initial branch name:
- `-shared` Initializes the repository with permissions suitable for sharing among multiple users. This sets `core.sharedRepository`, causing files under `$GIT_DIR` to be created with the requested permissions. If `-shared` is not specified, Git uses the normal permissions determined by your system `umask`.
- Tells Git where to initialize the repository. If the directory does not exist, Git creates it. If it does exist, Git initializes the repository inside it.

- **The areas of Git:** There are four important areas to be aware of when you are working with Git:

1. **Working directory:** The actual files and folders you see and edit in your project directory.
2. **Staging area:** The intermediate area where you choose which changes will go into the next commit.

The staging area is also called the index. When you run

```
1 git add main.c
```

you copy the current version of `main.c` from the working directory into the staging area.

This lets you prepare a commit carefully. For example, you might edit three files but stage only one of them.

You can inspect staged changes with:

```
0 git diff --staged
```

3. **Commit history:** The sequence of commits that record the project’s past states. A commit is a snapshot of the staged files, plus metadata such as author, date, commit message, and a pointer to the previous commit.

A — B — C

Here, *A*, *B*, and *C* are commits. The latest commit on your branch is usually what `HEAD` points to.

You can view the commit history with

```
1 git log
```

4. **Local repository:** The Git repository stored on your own machine. In a normal project, the local repository is mainly the .git directory

The .git/ directory contains Git's internal database: commits, branches, tags, object files, configuration, refs, and the staging area.

- **Working tree:** The working tree is the set of actual project files that git has checked out for you to edit. For

```
1 myproject/  
2   main.c  
3   README.md  
4   src/  
5   .git/
```

The working tree is

```
1 main.c  
2 README.md  
3 src/
```

- **git diff:** Git can show the difference between the working tree and the staging area with

```
1 git diff
```

- **Tracked files:** A tracked file is a file Git already knows about.

More precisely, a file is tracked if it is already in Git's repository history or currently in the index.

A tracked file is under Git's management. It can be in states such as:

1. unmodified
2. modified
3. staged
4. deleted

- **git add -A:** Stages all additions, modifications, and deletions in the whole repository.

```
1 git add -A
```

stages all changes in the entire repository, regardless of what directory you are currently in.

```
1 git add .
```

stages changes only in the current directory and below.

- **git restore, revert, and reset:**
 - **Restore:** git restore is mainly for undoing changes in your working tree or staging area. Use it when you want to throw away uncommitted changes.

```
1 git restore file.txt
```

This discards unstaged changes to file.txt in your working tree and restores it to the version currently in the staging area.

```
1 git restore --staged file.txt
```

```
1 git restore --worktree file.txt
```

discards unstaged changes in the file and restores it from the index. Note that this is equivalent to

```
1 git restore file.txt
```

because `--worktree` is the default target for git restore.

- **revert:** git revert undoes a commit by creating a new commit that applies the opposite changes.

Use it when the bad commit is already part of shared history, such as a branch you pushed to GitHub.

If commit **abc123** adds a line *"hello"*, then

```
1 git revert abc123
```

creates a new commit that removes that line. The original commit still exists in history.

- **Reset:** git reset moves the current branch pointer to another commit. Depending on the option, it may also change the staging area and working tree.

Use it when you want to move your branch backward or rewrite local history.

- **Branches:** A branch in Git is a movable name that points to a commit. That is the core idea. Suppose history looks like this:

A -- B -- C

Suppose main is a branch, and points to commit *C*. When you make a new commit while on main, Git creates a new commit and moves main forward:

Branches let you work on a separate line of development without disturbing another line.

```
1 git switch -c feature-login
```

This creates a new branch called feature-login and switches to it. Now, both main and feature-login point to commit *C*. If you commit on feature-login, only that branch moves forward.

```
A -- B -- C
           ^
           |
        main
           \
            D -- E
                  ^
            feature-login
```

- **HEAD:** HEAD means “where you currently are.” Usually, head points to a branch. If you make a commit, git moves the branch that HEAD is attached to

```
1 git branch feature-login
```

Creates a branch but does not switch to it.

```
1 git switch -c feature-login
2 git checkout -b feature-login
```

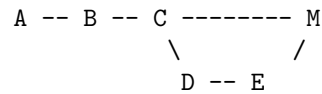
Both create the branch and switch to it

- **Merging branches:** Eventually, you usually want to bring the changes from one branch into another. Suppose you want to merge feature-login into main. First, switch to main.

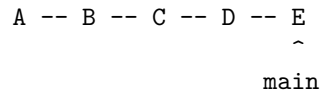
```
1 git switch main
```

Then, merge

```
1 git merge feature-login.
```



M is a merge commit. It combines the histories of `main` and `feature-login`. Sometimes Git can do a **fast-forward merge** instead:



After merging a branch, you can delete the local branch name:

```
1 git branch -d feature-login
```

This deletes the branch pointer, not necessarily the commits. If the commits are already reachable from another branch, such as `main`, they are still part of history.

```
1 git branch -D feature-login
```

This deletes the branch even if Git thinks it has unmerged work.

- **Local vs remote branches:** A local branch exists in your repository. A **remote-tracking branch** represents the state of a branch on a remote, such as GitHub.

`origin/main` means: My local record of what `main` looked like on the remote named `origin`.

```
1 git fetch
```

updates remote-tracking branches.

```
1 git push origin feature-login
```

Git sends your local `feature-login` branch to the remote.

- **git fetch:** The `git fetch` command downloads the latest commits, files, and references from a remote repository into your local repository, but it does not merge or integrate them into your current working code.

It essentially acts as a safe, "read-only" check that updates Git's internal tracking records to mirror what is happening on the server. Because it never alters your local branches or working files, it is completely safe to run at any time without risking merge conflicts.

- **Checkout vs switch:** Git checkout is the older, overloaded command. Switch is the newer, more specific command for switching branches.

```
1 git checkout main
2 git switch main
```

Both switch you to the main branch. But checkout can do multiple unrelated things. It can also restore files (seen below).

- **Hunks:** In Git, a hunk is a contiguous block of changes inside a file. When Git shows a diff, it does not usually show the entire file. It shows only the changed regions, plus a little surrounding context. Each changed region is called a **hunk**.

```
1 diff --git a/file.txt b/file.txt
2 index 83db48f..f735c2d 100644
3 --- a/file.txt
4 +++ b/file.txt
5 @@ -1,5 +1,5 @@
6     line 1
7     line 2
8 -old line
9 +new line
10    line 4
11    line 5
```

This part is the hunk:

```
1 @@ -1,5 +1,5 @@
2     line 1
3     line 2
4 -old line
5 +new line
6     line 4
7     line 5
```

- The -old line means that line was removed.
- The +new line means that line was added.

The unchanged lines around it are context, included so Git and humans can understand where the change belongs.

```
1 @@ -1,5 +1,5 @@
```

is the **hunk header**. It means:

```
1  -1,5    old file: starting at line 1, show 5 lines
2  +1,5    new file: starting at line 1, show 5 lines
```

So the hunk describes a change from one version of the file to another.

- **Patches:** A patch is a text representation of changes. In Git, a patch usually means a **diff file** that describes how to change one version of files into another version.

A patch is made out of one or more **hunks**.

You can create a patch with

```
1  git diff > changes.patch
```

Someone else can apply it with

```
1  git apply changes.patch
```

That modifies their working tree using the changes described in the patch.

- **Merges:** A merge in git combines changes from one branch into another branch.

```
1  git switch main
2  git merge feature
```

This means: Take the changes from feature and integrate them into main.

A merge conflict happens when Git cannot automatically combine the changes. Suppose both branches changed the same line differently,

```
0  // main.c
1  int x = 10;
```

```
1  // feature.c
2  int x = 20;
```

Git cannot possibly know which one you want, so it marks the conflict in the file.

```

1 <<<<<<< HEAD
2 int x = 10;
3 =====
4 int x = 20;
5 >>>>>> feature

```

This means

```

1 HEAD side:    your current branch, main
2 other side:   the branch being merged, feature

```

You manually edit the file to the final version you want, then you stage the resolved file, and finish the merge with a commit.

- **Cherry pick:** git revert and git cherry-pick are almost opposites in spirit.

git cherry-pick copies a commit's changes into your current branch. Use cherry-pick when you want to take one specific commit from another branch and apply it to your current branch.

- **Tags:** You can attach a name to a commit using a tag.

```

1 git tag v1.0 a3f4c91

```

Now, v1.0 refers to that commit.

- **Stashing:** Git stash temporarily saves your uncommitted changes somewhere inside git, then cleans your working tree. Use it when you are in the middle of work but need to switch branches, pull changes, or do something else without committing yet.

```

1 git stash

```

This takes your current uncommitted changes and stores them in a stash entry.

Then your working tree goes back to a clean state.

```

1 git stash list

```

lists the stashes

```

1 git stash apply

```

Brings back the most recent stash

```
1 git stash pop
```

Applies and removes from the list

```
1 git stash apply <stash name>
```

Applies a specific stash from the list.

```
1 git stash drop <name>
2 git stash clear
```

Removes stashes from the list, or clears the list.

```
1 git stash --staged
```

stashes only the staged changes. That means Git takes the changes currently in the index/staging area and puts them into a stash. Your unstaged working-tree changes are left alone.

```
1 git stash --keep-indexd
```

This stashes your unstaged changes, but keeps your staged changes in place.

In other words, the index is preserved.

- **Staging area manipulations:** Staging is represented by a directory **index** instead of **.git**. The commands that
 - **git add:** Stages file contents. More precisely, it copies the current content of selected working-directory files into the index, preparing that content for the next commit. `git add -p` stages selected hunks interactively. `git add -A` stages additions, modifications, and deletions.
 - **git rm:** Removes paths from the index. By default, it removes them from both the working tree and the index. With `--cached`, it removes them from the index only, leaving the file in your working directory

```
1 git rm file.c
2 git rm --cached file.c
```

Use `git rm --cached` when you accidentally started tracking a file but want to keep the local copy.

- **git mv:** Moves or renames a file and updates the index to record that change. The change is staged, but it still needs to be committed.

```
1 git mv oldname.c newname.c
```

This is roughly equivalent to

```
1 mv oldname.c newname.c
2 git rm oldname.c
3 git add newname.c
```

- **git restore --staged**: Changes the index without necessarily changing the working directory. This is the modern command for “unstaging” something. By default, `git restore --staged <path>` restores the index version from HEAD.

```
1 git restore --staged file.c
```

You can also restore both the index and working tree:

```
0 git restore --staged --worktree file.c
```

- **git reset**: `git reset` can move HEAD, reset the index, and sometimes reset the working tree depending on the mode.

```
1 git reset file.c
```

Unstages file.c

```
1 git reset
```

Resets the whole index to HEAD, leaving the working directory alone

```
1 git reset --mixed HEAD~1
```

Moves HEAD and resets the index, but keeps working-directory changes. `--mixed` is the default reset mode.

```
1 git reset --hard HEAD
```

Resets both the index and working tree to HEAD

```
0 git reset --soft HEAD
```

Resets the working tree but **not** the index.

- **git checkout**: git checkout is older and overloaded. It can affect the index in some forms but not others.

This form updates both the working tree and the index from another commit/tree:

```
1 git checkout main -- file.c
```

That means: take file.c from main, put it into my working tree, and stage that version in the index.

Recall that -- signals the end of command options.

The form

```
1 git checkout -- file.c
```

Restores the working-tree file from the index, so it affects the working directory, not the staging area.

- **git switch**: Switching branches updates the working tree and the index to match the branch being checked out

```
1 git switch main
2 git switch feature
```

So while git switch is not normally thought of as a staging command, it does replace the index as part of changing branches

- **git commit**: A plain commit reads the current index and creates a commit from it. The Git docs describe git commit as creating a new commit from the current contents of the index

```
1 git commit -a
```

Automatically stages modifications and deletions for already-tracked files before committing. It does not add brand-new untracked files.

```
1 git commit -p
```

Lets you interactively choose hunks to include in the commit.

```
1 git commit file.c
```

Commits the current content of the named tracked file, bypassing the normal “only what is already staged” behavior for that file.

- **git apply --index**: Applies a patch to both the working tree and the index.

```
1 git apply --index patch.patch
```

- **git apply --cached**: Applies a patch only to the index, leaving the working tree alone

```
1 git apply --cached patch.patch
```

Without `--index` or `--cached`, `git apply` applies the patch only to files in the working tree.

- **git merge**: Merging updates the index and working tree when changes apply cleanly. If conflicts occur, the index stores special unmerged entries until you resolve them and stage the resolved files.

```
1 git merge feature
```

After conflict resolution,

```
1 git add conflicted-file.c
2 git commit
```

- **git cherry-pick**: Applies the changes from an existing commit. If paths apply cleanly, they are updated in both the index and working tree. If conflicts occur, the index records multiple versions for conflicted paths.

```
1 git cherry-pick abc123
```

After resolving conflicts:

```
1 git add file.c
2 git cherry-pick --continue
```

- **git revert**: Creates a new commit that reverses an earlier commit. With `--no-commit/-n`, it applies the reverse changes to the working tree and index without committing.
- **git rebase**: Rebases repeatedly applies commits onto a new base. Internally, this affects the working tree and index as commits are replayed. If conflicts occur, you resolve files, stage them with `git add`, and continue:

```
1 git rebase main
2 git add file.c
3 git rebase --continue
```

- **git pull**: git pull is essentially git fetch followed by either git merge or git rebase, depending on configuration/options. Because the merge/rebase part can update the index, git pull can affect the staging area indirectly.

```
1 git pull
2 git pull --rebase
```

- **git stash**: git stash records the current state of both the working directory and the index, then reverts the working directory to match HEAD.

```
1 git stash
2
3 git stash --staged
4 git stash --keep-index
5 git stash pop
6 git stash apply
```

git stash pop and git stash apply can also modify the index depending on how the stash was created and applied.

- **Low-level index manipulations:**

- **git update-index**: This is the direct plumbing command for modifying the index. It can add entries, remove entries, refresh stat information, change executable bits, set assume-unchanged, set skip-worktree, and more. Git's own documentation says git update-index "modifies the index."

```
1 git update-index --chmod=+x script.sh
2 git update-index --assume-unchanged config.local
3 git update-index --skip-worktree config.local
```

- **git read-tree**: Low-level command for reading tree objects into the index. It is used internally by commands such as checkout, merge, and reset-like operations.

```
1 git read-tree HEAD
```

- **git ls-files**: git ls-files lists files that Git knows about in the index/staging area, and optionally files in the working tree

```
1 git ls-files
```

This prints the files that are currently tracked by Git, because tracked files are recorded in the index.


```
1 git ls-files --others
```

Shows untracked files

```
1 git ls-files --others --exclude-standard
```

Shows untracked files, but respect .gitignore.

```
1 git ls-files --ignored --others --exclude-standard
```

```
1 git ls-files --deleted
```

Shows deleted tracked files

```
1 git ls-files --modified
```

Shows modified tracked files

```
1 git ls-files --unmerged
```

Shows files with merge conflicts.

- **git status:** Shows the current state of your repo relative to the current commit, usually HEAD.
- **git log:** Shows the commit history

```
1 git log
```

Show recent commits only:

```
1 git log -5
```

Show commits affecting a specific file:

```
1 git log -- main.c
```

Show patches introduced by each commit:

```
1 git log -p
```

Show summary of changed files:

```
1 git log --stat
```

- **Tracking:** In Git, to **track** means: A local branch is linked to a remote branch so Git knows which remote branch it should compare, pull from, and push to by default.
- **Upstreams:** An upstream is the remote branch that your local branch is configured to “track.”

For example, suppose you have:

```
1 main
2 origin/main
```

If your local `main` is tracking `origin/main`, then `origin/main` is the upstream of `main`. You can see this with

```
1 git branch -vv
```

```
1 * main abc1234 [origin/main] Add parser notes
```

So, local branch `main` has upstream `origin/main`. The upstream matters because commands like these use it automatically.

```
1 git pull
2 git push
3 git status
```

If your branch has an upstream, then

```
1 git push
```

usually means

```
1 git push origin main
```

Also,

```
1 git pull
```

usually means

```
1 git fetch origin
2 git merge origin/main
```

- **Setting an upstream:** When you first push a local branch, you often use

```
1 git push -u origin main
```

Or, equivalently

```
1 git push --set-upstream origin main
```

The `-u` means *set upstream*.

You can also manually set an upstream:

```
1 git branch --set-upstream-to=origin/main main
```

Or, if already on main,

```
1 git branch --set-upstream-to=origin/main
```

- **Upstreams and branches:** You do not always need to set an upstream for a branch.

You only need an upstream if you want Git to know which remote branch your local branch corresponds to for commands like:

```
1 git pull
2 git push
3 git status
```

Suppose you create a local branch

```
1 git switch -c feature
```

At this point, `feature` is just local. It does not automatically have an upstream unless you created it from a remote branch in a tracking-aware way.

If you only want to work locally, no upstream is needed.

But if you want to push it, you usually do

```
1 git push -u origin feature
```

This does two things

1. Creates / updates the remote branch

```
1 origin/feature
```

2. Sets your local branch **`feature`** to track **`origin/feature`**.

- **Remote-tracking branch:** A remote-tracking branch is Git's local record of a branch on a remote.

```
1 origin/main
2 origin/dev
3 origin/feature-x
```

These are not branches you usually edit directly. They are Git's local snapshots of what the remote branches looked like the last time you fetched.

An **upstream** is a relationship between your local branch and one of those remote-tracking branches.

- **Rebase:** A rebase moves your commits so that they appear to be based on a different commit. Suppose we have history

```
A---B---C  origin/main
      \
      D---E  feature
```

You made commits *D* and *E* on `feature`, but meanwhile `main` advanced to *C*. If you run

```
1 git switch feature
2 git rebase origin/main
```

Git rewrites your branch like this:

```
A---B---C  origin/main
      \
      D'---E' feature
```

Your changes are replayed on top of the latest origin/main.

Notice the commits are now called D' and E' , not D and E . That is because rebasing creates new commits with new commit hashes.

- **Rebase vs merge:** Both merge and rebase can bring one branch up to date with another branch, but they do it differently.
 - **Merge:**

```
1 git merge origin/main
```

creates a new merge commit:

```

A---B---C  origin/main
 \       \
  D---E---M  feature
```

The history preserves the fact that the branches diverged and were merged.

- **Rebase:**

```
1 git rebase origin/main
```

rewrites the feature branch

```

A---B---C  origin/main
 \
  D'---E'  feature
```

The history looks linear, as if you started your feature branch from the latest main. Rebase is commonly used to keep a clean, linear history.

For example, before merging a feature branch, you might do:

```
1 git fetch origin
2 git rebase origin/main
```

This updates your feature branch so your commits are placed after the newest commits on main.

- **Rebasing with upstream:** If your branch has an upstream, you can do:

```
1 git pull --rebase
```

This means:

```
1  git fetch
2  git rebase <upstream>
```

So instead of fetching and merging the upstream branch, Git fetches and rebases your local commits on top of it.

- **Blobs:** a blob is the object type that stores the contents of a file.

A blob does not store the file name, path, permissions, or directory structure. It only stores the raw file data.

If you have

```
1  src/main.c
```

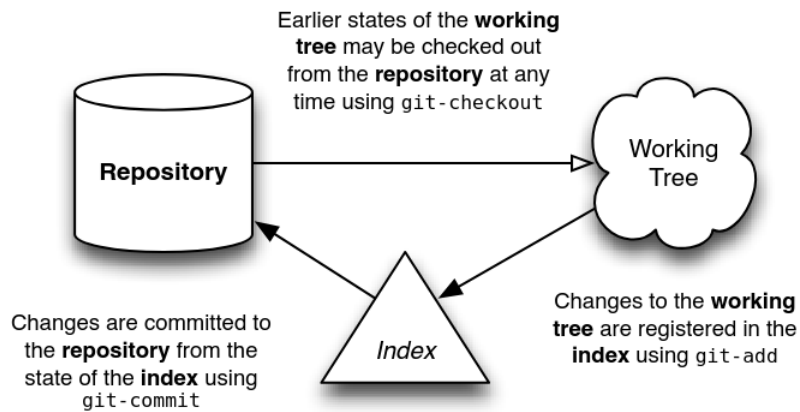
Git stores the contents of main.c as a **blob**. The filename main.c and the path src/main.c are stored separately in a **tree object**.

- **blob:** file contents
- **tree:** directory listing: names + permissions + pointers to blobs/trees
- **commit:** snapshot metadata + pointer to a tree

- **Summary:**

- **repository:** A repository is a collection of commits, each of which is an archive of what the project's working tree looked like at a past date, whether on your machine or someone else's. It also defines HEAD (see below), which identifies the branch or commit the current working tree stemmed from. Lastly, it contains a set of branches and tags, to identify certain commits by name.
- **index:** Unlike other, similar tools you may have used, Git does not commit changes directly from the working tree into the repository. Instead, changes are first registered in something called the index. Think of it as a way of "confirming" your changes, one by one, before doing a commit (which records all your approved changes at once). Some find it helpful to call it instead as the "staging area", instead of the index.
- **working tree:** A working tree is any directory on your filesystem which has a repository associated with it (typically indicated by the presence of a sub-directory within it named .git.). It includes all the files and sub-directories in that directory.
- **commit:** A commit is a snapshot of your working tree at some point in time. The state of HEAD (see below) at the time your commit is made becomes that commit's parent. This is what creates the notion of a "revision history".
- **branch:** A branch is just a name for a commit (and much more will be said about commits in a moment), also called a reference. It's the parentage of a commit which defines its history, and thus the typical notion of a "branch of development".
- **tag:** A tag is also a name for a commit, similar to a branch, except that it always names the same commit, and can have its own description text.

- **master**: The mainline of development in most repositories is done on a branch called “master”. Although this is a typical default, it is in no way special.
- **HEAD**: HEAD is used by your repository to define what is currently checked out:
 - * If you checkout a branch, HEAD symbolically refers to that branch, indicating that the branch name should be updated after the next commit operation.
 - * If you checkout a specific commit, HEAD refers to that commit only. This is referred to as a detached HEAD, and occurs, for example, if you check out a tag name.



Repository: Directory content tracking

- **Data is immutable:** Once Git stores an object in the repository, Git does not modify that object in place. Instead, if something changes, Git creates a **new object** with a **new hash**.

Git's object database stores objects such as:

- **blob** : file contents
- **tree** : directory structure
- **commit** : snapshot metadata + pointer to a tree
- **tag** : annotated tag object

Each object is identified by a hash of its contents. For example, a blob's ID is computed from the blob's data plus a small header.

$$\text{hash}(\text{contents}) = \text{object ID}.$$

So, if the contents change, the hash changes.

```
1  echo "hello" | git hash-object --stdin
2  echo "hello world" | git hash-object --stdin
```

Those produce different object IDs because the contents are different. That means Git cannot just “edit” an existing blob. If it did, the object's contents would no longer match its hash. The old object ID would become invalid.

Git does this

```
1  blob abc123: "hello"
2  blob def456: "hello world"
```

The old object still exists. The new version is stored as a separate object. The same idea applies to commits. A commit contains data like:

```
1  tree pointer
2  parent commit pointer
3  author
4  committer
5  message
6  timestamp
```

If any of that changes, the commit hash changes. So when you “amend” a commit, Git does not actually edit the old commit. It creates a new commit object.

- **Blobs and trees:** A blob represents the contents of one file at one particular version.

A **Git tree** is basically Git’s representation of a directory. A tree object contains entries. Each entry records

mode	name	object id
------	------	-----------

For files, the object ID points to a blob

100644	main.c	blob abc123	100644	README.md	blob def456
--------	--------	-------------	--------	-----------	-------------

For subdirectories, the object ID points to another **tree**.

- **Object modes:** In Git, object modes are the mode values stored in tree objects to describe what kind of entry each path is.

Mode	Meaning
100644	Regular file, not executable
100755	Regular file, executable
040000	Directory, represented by a tree object
120000	Symbolic link
160000	Git submodule, also called a gitlink

- **Intro:** What Git does is quite rudimentary: it maintains snapshots of a directory’s contents. Much of its internal design can be understood in terms of this basic task.

The design of a Git repository in many ways mirrors the structure of a UNIX filesystem: A filesystem begins with a root directory, which typically consists of other directories, most of which have leaf nodes, or files, that contain data. Meta-data about these files’ contents is stored both in the directory (the names), and in the i-nodes that reference the contents of those file (their size, type, permissions, etc). Each i-node has a unique number that identifies the contents of its related file. And while you may have many directory entries pointing to a particular i-node (i.e., hard-links), it’s the i-node which “owns” the contents stored on your filesystem.

Internally, Git shares a strikingly similar structure, albeit with one or two key differences. First, it represents your file’s contents in blobs, which are also leaf nodes in something awfully close to a directory, called a tree. Just as an i-node is uniquely identified by a system-assigned number, a blob is named by computing the SHA1 hash id of its size and contents. For all intents and purposes this is just an arbitrary number, like an i-node, except that it has two additional properties: first, it verifies the blob’s contents will never change; and second, the same contents shall always be represented by the same blob, no matter where it appears: across commits, across repositories — even across the whole Internet. If multiple trees reference the same blob, this is just like hard-linking: the blob will not disappear from your repository as long as there is at least one link remaining to it.

The difference between a Git blob and a filesystem’s file is that a blob stores no metadata about its content. All such information is kept in the tree that holds the blob. One tree may know those contents as a file named “foo” that was created in August 2004, while another tree may know the same contents as a file named “bar” that

was created five years later. In a normal filesystem, two files with the same contents but with such different metadata would always be represented as two independent files. Why this difference? Mainly, it's because a filesystem is designed to support files that change, whereas Git is not. The fact that data is immutable in the Git repository is what makes all of this work and so a different design was needed. And as it turns out, this design allows for much more compact storage, since all objects having identical content can be shared, no matter where they are.

Note: Git tracks commits. A commit points to a tree, the tree maps filenames to blobs, and the blobs store file contents.

- **The blob:** We can create a simple example,

```
1 mkdir sample && cd sample && echo "Hello, world!" > greeting
2 git hash-object greeting # ->
   ↪ af5626b4a114abcb82d63db7c8082c3c4756e51b
```

No matter where you run `git hash-object`, if the file has only the content *Hello, world!*, the hash will be the same.

If another repository has a commit whose tree refers to the same blob object, then when we fetch or pull from that repository, Git can recognize that we already have that blob because its object ID is the same. Git only needs to download/store objects that we do not already have.

If we now create a repository,

```
1 git init
2 git add greeting
3 git commit -m "Added greeting"
```

Our blob should be in the system exactly as we expected, using the hash id determined above. As a convenience, Git requires only as many digits of the hash id as are necessary to uniquely identify it within the repository. Usually just six or seven digits is enough:

```
1 git cat-file -t af5626b # -> blob
2 git cat-file blob af5626b # -> Hello, world!
```

a Git blob represents the fundamental data unit in Git. Really, the whole system is about blob management.

- **Blobs are stored in trees** In order for Git to represent the structure and naming of your files, it attaches blobs as leaf nodes within a tree. Now, I can't discover which tree(s) a blob lives in just by looking at it, since it may have many, many owners. But I know it must live somewhere within the tree held by the commit I just made:

```
1  git ls-tree HEAD # -> 100644 blob
   ↳ af5626b4a114abcb82d63db7c8082c3c4756e51b greeting
```

This commit contains one Git tree, which has a single leaf: the greeting content's blob.

Although I can look at the tree containing my blob by passing HEAD to ls-tree, I haven't yet seen the underlying tree object referenced by that commit. Here are a few other commands to highlight that difference and thus discover my tree:

```
1  git rev-parse HEAD # ->
   ↳ 0437733255614428faae0e090712264cf336cd37
2  git cat-file -t HEAD # -> commit
3  git cat-file commit HEAD
4
5  tree 0563f77d884e4f79ce95117e2d686d7d6e282887
6  author hgoose <noodlecloudcode@gmail.com> 1780326086 -0500
7  committer hgoose <noodlecloudcode@gmail.com> 1780326086 -0500
8
9  b # -> commit message
```

The first command decodes the HEAD alias into the commit it references, the second verifies its type, while the third command shows the hash id of the tree held by that commit, as well as the other information stored in the commit object. The hash id for the commit is unique to my repository — because it includes my name and the date when I made the commit — but the hash id for the tree should be common between your example and mine, containing as it does the same blob under the same name.

```
1  git ls-tree 0563f77 # -> 100644 blob
   ↳ af5626b4a114abcb82d63db7c8082c3c4756e51b greeting
```

There you have it: my repository contains a single commit, which references a tree that holds a blob — the blob containing the contents I want to record. There's one more command I can run to verify that this is indeed the case:

```
1  find .git/objects -type f | sort
2  .git/objects/04/37733255614428faae0e090712264cf336cd37
3  .git/objects/05/63f77d884e4f79ce95117e2d686d7d6e282887
4  .git/objects/af/5626b4a114abcb82d63db7c8082c3c4756e51b
```

We see three objects. These objects should refer to a tree, a commit, and a blob. Let's take a look

```

1 git cat-file -t 0437733255
2 commit
3 git cat-file -t 0563f77
4 tree
5 git cat-file -t af5626b
6 blob

```

Notice the directory name is the MSB of each hash.

```

1 git show 0437733
2 commit 0437733255614428faae0e090712264cf336cd37 (HEAD ->
   ↪ main)
3 Author: hgoose <noodlecloudcode@gmail.com>
4 Date:   Mon Jun 1 10:01:26 2026 -0500
5
6 b
7
8 diff --git a/greeting b/greeting
9 new file mode 100644
10 index 0000000..af5626b
11 --- /dev/null
12 +++ b/greeting
13 @@ -0,0 +1 @@
14 +Hello, world!
15
16 git show 0563f77
17 tree 0563f77
18
19 greeting
20
21 git show af5626b
22 Hello, world!

```

- **How trees are made:** Every commit holds a single tree, but how are trees made? We know that blobs are created by stuffing the contents of your files into blobs — and that trees own blobs — but we haven't yet seen how the tree that holds the blob is made, or how that tree gets linked to its parent commit.

Let's start with a new sample repository again, but this time by doing things manually, so you can get a feeling for exactly what's happening under the hood:

```

1 rm -rf greeting .git
2 echo "Hello, world!" > greeting
3 git init
4 git add greeting

```

It all starts when you first add a file to the index.

```

1 git log # Fails, no commits
2
3 git ls-files --stage
4 100644 af5626b4a114abcb82d63db7c8082c3c4756e51b 0 greeting

```

What's this? I haven't committed anything to the repository yet, but already an object has come into being. It has the same hash id I started this whole business with, so I know it represents the contents of my greeting file. I could use `cat-file -t` at this point on the hash id, and I'd see that it was a blob. It is, in fact, the same blob I got the first time I created this sample repository.

This blob isn't referenced by a tree yet, nor are there any commits. At the moment it is only referenced from a file named `.git/index`, which references the blobs and trees that make up the current index. So now let's make a tree in the repo for our blob to hang off of:

```

1 git write-tree # Records the contents of the index in a tree
2 0563f77d884e4f79ce95117e2d686d7d6e282887

```

This number should look familiar as well: a tree containing the same blobs (and subtrees) will always have the same hash id. I don't have a commit object yet, but now there is a tree object in that repository which holds the blob. The purpose of the low-level `write-tree` command is to take whatever the contents of the index are and tuck them into a new tree for the purpose of creating a commit.

I can manually make a new commit object by using this tree directly, which is just what the `commit-tree` command does:

```

1 git commit-tree 0563f77 -m "Initial commit"
2 d63d26194dd7a5cfbb9fe7365874c2918707f091

```

The raw `commit-tree` command takes a tree's hash id and makes a commit object to hold it. If I had wanted the commit to have a parent, I would have had to specify the parent commit's hash id explicitly using the `-p` option.

Our work is not done yet, though, since I haven't registered the commit as the new head of the current branch:

```

1 echo d63d26194dd7a5cfbb9fe7365874c2918707f091 >
   ↪ .git/refs/head/master

```

This command tells Git that the branch name "master" should now refer to our recent commit. Another, much safer way to do this is by using the command `update-ref`

```

1 git update-ref refs/heads/master d63d2619

```

After creating master, we must associate our working tree with it. Normally this happens for you whenever you check out a branch.

```
1 git symbolic-ref HEAD refs/heads/master
```

This command associates HEAD symbolically with the master branch. This is significant because any future commits from the working tree will now automatically update the value of refs/heads/master.

The command `git symbolic-ref HEAD refs/heads/master` manually forces Git's HEAD pointer to point directly to the master branch without altering your working directory or changing your files. It writes the string `ref: refs/heads/master` straight into `.git/HEAD`.

It's hard to believe it's this simple, but yes, I can now use `log` to view my newly minted commit

```
1 git log
```

A side note: if I hadn't set `refs/heads/master` to point to the new commit, it would have been considered "unreachable", since nothing currently refers to it nor is it the parent of a reachable commit. When this is the case, the commit object will at some point be removed from the repository, along with its tree and all its blobs. (This happens automatically by a command called `gc`, which you rarely need to use manually). By linking the commit to a name within `refs/heads/`, as we did above, it becomes a reachable commit, which ensures that it's kept around from now on.

- **The beauty of commits:**